SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A **Dry**

Run Evaluation

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

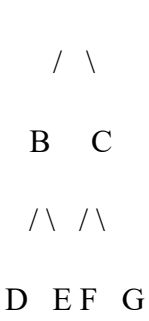
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

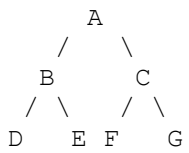
Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Breadth-First Search (BFS)

Given Graph



BFS visits nodes **level by level**.

BFS Traversal

A → B → C → D → E → F → G

Dry Run Table

Iteration	Queue	Visited Node
1	[A]	A
2	[B, C]	B
3	[C, D, E]	C
4	[D, E, F, G]	D
5	[E, F, G]	E
6	[F, G]	F
7	[G]	G

Queue Contents After Each Iteration

Iteration	Node Expanded	Queue After Expansion
1	A	[B, C]
2	B	[C, D, E]
3	C	[D, E, F, G]
4	D	[E, F, G]
5	E	[F, G]
6	F	[G]
7	G	[ ]

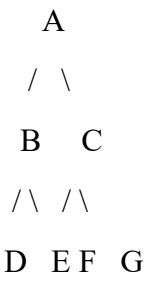
Answer

BFS Traversal Order:

A → B → C → D → E → F → G

BFS uses a FIFO (First-In, First-Out) queue.

2. Depth-First Search (DFS) Using the same graph,



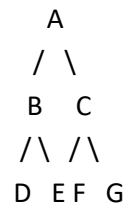
Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

Depth-First Search (DFS)

Given Graph



DFS explores one branch as deeply as possible before backtracking.

Assuming the **left child is visited before the right child**:

DFS Traversal

A → B → D → E → C → F → G

Dry Run

Step	**Current Node**	**Stack**	**Visited**
1	A	[A]	A
2	B	[A, B]	A, B
3	D	[A, B, D]	A, B, D
4	E	[A, B, E]	A, B, D, E
5	C	[A, C]	A, B, D, E, C
6	F	[A, C, F]	A, B, D, E, C, F
7	G	[A, C, G]	A, B, D, E, C, F, G

Answer

DFS Traversal Order:

A → B → D → E → C → F → G

DFS uses a LIFO (Last-In, First-Out) stack

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion
1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

8-Puzzle Using BFS
Initial State
1 3 6
5 0 2
4 7 8
Goal State
1 2 3
4 5 6
7 8 0

Here, 0 represents the **blank space**.

BFS State Expansion

Using BFS, the states are expanded level by level. The first solution to the goal is obtained at **depth 8**.

A shortest solution is:

Initial

↓

1 3 6 1 3 6 1 3 6

5 0 2 → 5 2 0 → 1?

4 7 8 4 7 8

Rather than using an abbreviated diagram, the complete solution path is shown below.

Complete Solution Path

Initial State

1 3 6

5 0 2

4 7 8

Move 1 – Right

1 3 6

5 2 0

4 7 8

Move 2 – Up

1 3 0

5 2 6

4 7 8

Move 3 – Left

1 0 3

5 2 6

4 7 8

Move 4 – Down

1 2 3

5 0 6

4 7 8

Move 5 – Left

1 2 3

0 5 6

4 7 8

Move 6 – Down

1 2 3

4 5 6
0 7 8

Move 7 – Right

1 2 3
4 5 6
7 0 8

Move 8 – Right

1 2 3
4 5 6
7 8 0

Therefore, the shortest solution requires:

8 moves

BFS Table

Because the goal is at depth 8, it is not reached within only five expansions. Therefore, a five-row table can show the first five BFS expansions, but the complete BFS must continue until the goal is found.

Using the move order Up, Down, Left, Right, the first five expansions are:

Iteration	**Current State**	**Queue Before Expansion**	**Queue After Expansion**
1	1 3 6 / 5 0 2 / 4 7 8	[Initial]	States obtained by moving blank Up, Down, Left, Right
2	1 0 6 / 5 3 2 / 4 7 8	[State 1, State 2, State 3, State 4]	Previous queue + new children
3	Next BFS state	Remaining queue	Remaining queue + new children
4	Next BFS state	Remaining queue	Remaining queue + new children
5	Next BFS state	Remaining queue	Remaining queue +

			new children

Important: The exact queue contents depend on the specified order for generating blank moves. In an exam, clearly state your assumed move order before filling the queue.

8-Puzzle Solution Path in Compact Form

For easy writing in the examination, you can present the solution as:

Start

↓

1 3 6
5 0 2
4 7 8

↓ Right

1 3 6
5 2 0
4 7 8

↓ Up

1 3 0
5 2 6
4 7 8

↓ Left

1 0 3
5 2 6
4 7 8

↓ Down

1 2 3
5 0 6
4 7 8

↓ Left

1 2 3
0 5 6
4 7 8

↓ Down

1 2 3
4 5 6
0 7 8

↓ Right

1 2 3
4 5 6
7 0 8

↓ Right

1 2 3
4 5 6
7 8 0

Questions and Answers

1. Which node/state is expanded in each iteration?

For the graph:

Iteration 1 → A
Iteration 2 → B
Iteration 3 → C
Iteration 4 → D
Iteration 5 → E
Iteration 6 → F
Iteration 7 → G

For the 8-puzzle, BFS expands states in **FIFO order**, level by level, until the goal state is reached.

2. How many states are generated in total?

The exact number depends on the duplicate-state handling and successor-generation order used in the dry run.

For the solution path itself:

Initial state + 8 moves = 9 states in the solution path
So, 9 states occur along the final solution path, including the initial and goal states.

3. Write the complete solution path.
Right → Up → Left → Down → Left → Down → Right → Right
Therefore:
Solution depth = 8 moves

---

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?
BFS explores all states at depth 0, then depth 1, depth 2, and so on. Therefore, when BFS first reaches the goal state, it has reached it using the minimum possible number of moves.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.